

MonoSQL部署指南

MonoSQL产品介绍

MonoSQL是[成章数据](#)公司自主设计、研发的基于DynamoDB的封装器，是一款可以帮助用户从Mysql或者MariaDB迁移到DynamoDB的虚拟镜像产品，具备无状态，自动扩容，数据安全等优点，借助MonoSQL，您可以继续使用JDBC或ODBC协议在DynamoDB实现数据的存储、检索和更新等操作，而不需更改原先的引用程序。

产品功能特性

MonoSQL server是无状态的，所有的目录和数据都存储在DynamoDB中。SQL查询将被MonoSQL服务器解析、优化和执行，而MonoSQL服务器将通过使用GetItem、PutItem等请求向DynamoDB请求实际数据。MonoMonitor实现对整个Auto Scaling组的监控，AWS Cloud Watch实现整个Auto Scaling组日志的存储。

- 用户和权限管理。使用 CREATE USER 命令在 MonoSQL 中管理用户。使用 GRANT 命令管理权限。两者都与 MySQL 兼容。
- DDL 操作。使用 CREATE TABLE 和 DROP TABLE 来管理表结构。
- DML 操作。支持 SELECT、INSERT、UPDATE、DELETE 命令。
- 连接操作符。
- 聚合操作符。
- CTE 和递归 CTE 操作符。

相比于Mysql与MariaDB，DynamoDB具有如下特点：

- 高扩展性：DynamoDB是一种高度可扩展的NoSQL数据库，可以轻松地扩展到数十亿行数据和高并发访问，而MySQL或MariaDB则需要复杂的集群架构和调优才能达到类似的规模和性能。
- 无服务器架构：DynamoDB是AWS提供的一种无服务器数据库，可以免去维护数据库服务器的繁琐任务，让开发者能够专注于应用程序的开发和部署。
- 高性能：DynamoDB是一种高性能的数据库，支持毫秒级别的响应时间和数百万次的并发访问。这使得它非常适合处理实时应用程序和高并发负载，而MySQL或MariaDB则需要进行复杂的优化和调整才能达到类似的性能。
- 弹性伸缩：DynamoDB支持弹性伸缩，可以根据实际负载自动调整数据库实例的数量和配置。这使得它非常适合处理不稳定的负载和峰值访问，而MySQL或MariaDB则需要进行手动调整和管理。
- 可用性和耐用性：DynamoDB具有高可用性和耐用性，可以保证数据不会丢失或损坏。这使得它非常适合处理重要的数据和应用程序，而MySQL或MariaDB则需要进行复杂的备份和恢复操作才能达到类似的效果。

从MySQL或MariaDB到DynamoDB的迁移最大的**难点与痛点**在于：由于DynamoDB和MySQL/MariaDB具有不同的API和查询语言，因此需要相应地修改应用程序和代码，以便能够正确地访问DynamoDB。MonoSQL可以帮助您直接使用原来的程序完成上述的数据访问操作，极大的节省企业或个人的转型成本。

产品部署概述

初始化MonoSQL在DynamoDB中创建系统表

DynamoDB是有AWS提供的NoSQL数据库服务，它可以快速、可靠地存储和检索数据。借助MonoSQL，您可以无痛地从Mysql或者MariaDB的轻松迁移到DynamoDB。在使用MonoSQL进行DynamoDB的访问之前需要先在DynamoDB上创建系统表，系统表是DynamoDB中的一种特殊表格，用于存储与DynamoDB服务本身相关的元数据和配置信息。通过运行初始化程序来创建系统表格，可以确保DynamoDB服务能够正常运行并提供所需的功能。

1. 基于MonoSQL的EC2实例创建

2. 在DynamoDB中创建MonoSQL对应的系统表,以ssh登陆到新建的EC2实例运行下面的命令，初始化MonoSQL在AWS DynamoDB中创建系统表格

```
bash /home/ubuntu/install/scripts/mysql_install_db --defaults-  
file=/home/ubuntu/dynosql.cnf --basedir=/home/ubuntu/install --  
datadir=/home/ubuntu/data0 --plugin-dir=/home/ubuntu/install/lib/plugin >  
log 2>&1 &
```

3. 启动数据库服务
- ```
bash /home/ubuntu/install/bin/mysqld --defaults-
file=/home/ubuntu/dynosql.cnf > mysql_log 2>&1 &
```

4. 使用sock连接数据库
- ```
bash # connect to db. cd ~/install sudo ./bin/mysql -S  
/tmp/mysqld3306.sock test
```

5. 通过下面的命令创建SQL用户sysb进行基准测试，创建用户mono用于数据库监控。 ``sql # create sysbench user and monitor user. delete from mysql.user where User=''; CREATE USER 'sysb'@'%' IDENTIFIED BY 'sysb'; GRANT ALL PRIVILEGES ON *.* TO 'sysb'@'%';

```
CREATE USER IF NOT EXISTS 'mono'@'%' IDENTIFIED BY 'mono' WITH  
MAX_USER_CONNECTIONS 5;  
GRANT ALL PRIVILEGES ON *.* TO 'mono'@'%' IDENTIFIED BY 'mono' WITH  
GRANT OPTION;  
FLUSH PRIVILEGES;  
````
```

创建上述角色后，可以进入`mysql`数据库下，查看`user`表中，是否存在用户`sysb`与`mono`，验证创建是否成功  


### 基于MonoSQL创建Auto Scaling Group

Auto Scaling Group是AWS提供的一种云端服务，它允许您自动地调整EC2实例的数量，以满足应用程序的需求。借助Auto Scaling Group，您可以实现根据请求流量自动配置MonoSQL的实例数量，而无需手动干预。选择 **EC2 > Auto Scaling > Auto Scaling组**，进入如下界面 点击**创建Auto Scaling组**，进入后续的Auto

Scaling Group具体配置。

Auto Scaling 组 (5) Info

搜索您的 Auto Scaling 组

创建 Auto Scaling 组

| <input type="checkbox"/> | 名称                                   | 启动模板/配置                                  | 实例 | 状态 | 所需容量 | 最... |
|--------------------------|--------------------------------------|------------------------------------------|----|----|------|------|
| <input type="checkbox"/> |                                      | MonoSQLConfZean                          | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | d4c3c020-0197-ff2a-2c3f-02dec84baeb6 | eks-d4c3c020-0197-ff2a-2c3f-02dec84baeb6 | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | 0b66-fd89-7a99-0b5de5088262          | eks-f8c21b8e-0b66-fd89-7a99-0b5de5088262 | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | 07a9-1876-0d59-0bb44ccd3aa7          | eks-c4c20ef1-07a9-1876-0d59-0bb44ccd3aa7 | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | e064-f04c-8e32-9d47dc060b4b          | eks-bcc1acc6-e064-f04c-8e32-9d47dc060b4b | 0  | -  | 0    | 0    |

1. 步骤1 选择启动模板或配置

- 设置Auto Scaling组名称，注意此处的组名称必须设置为MonoSQL，因为后续的监控组件Prometheus与Grafana会依据此名称来对该资源组进行监控。

名称

Auto Scaling 组名称  
输入一个名称来标识该组。  
MonoSQL  
名称对于当前区域中的此账户必须是唯一的，且不超过 255 个字符。

- 设置启动模板，启动模板选择启动配置，忽略警告项，选择创建启动配置

启动配置 Info

切换到启动模板

我们建议您使用启动模板并使用 Auto Scaling 指导选项，而不是使用启动配置来创建 EC2 自动扩缩组。有关迁移启动配置和使用启动模板的更多信息，请参阅文档

启动配置  
选择一个包含 Amazon Machine Image (AMI)、实例类型、密钥对和安全组等实例级别设置的启动配置。  
选择启动配置  
必填  
创建启动配置

取消

下一步

按照如下的演示，配置自定义的启动模板

EC2 > 启动配置 > 创建启动配置

### 创建启动配置 信息

⚠ 我们建议您使用启动模板并使用自动扩缩指引选项，而不是使用启动配置来创建 EC2 Auto Scaling 组。有关迁移启动配置和使用启动模板的更多信息，[请参阅文档](#)

[创建启动模板](#)

#### 启动配置名称

名称

MonoSQLConfAlpha

#### Amazon 系统映像 (AMI) 信息

AMI

### 注意

在配置启动模板的时候，设置的IAM实例配置文件需要满足[部署准备](#)中的条件2，即选择已经创建好的名称为**monosql**的用户角色。

#### 其他配置 - 可选

购买选项 信息

☐ 请求 Spot 实例

IAM 实例配置文件 信息

monosql

监控 信息

☐ 启动 CloudWatch 中的 EC2 实例详细监控

2. **步骤2** 选择实例启动选项 选择相应的可用区与子网为ap-northeast-1a | subnet-015bc66fedce1b8c0

## 选择实例启动选项 Info

选择您的实例启动到的 VPC 网络环境，然后自定义实例类型和购买选项。

### 网络 Info

对于大多数应用程序，您可以使用多个可用区，并让 EC2 Auto Scaling 跨可用区平衡实例。默认 VPC 和默认子网适合快速入门。

#### VPC

选择为您的 Auto Scaling 组定义虚拟网络的 VPC。

vpc-0dd5ac199e158931f  
172.31.0.0/16 Default



[创建 VPC](#)

#### 可用区和子网

定义您的 Auto Scaling 组可以在所选 VPC 中使用哪些可用区和子网。

选择可用区和子网



ap-northeast-1a | subnet-015bc66fedce1b8c0 X  
172.31.32.0/20 Default

[创建子网](#)



### 3. 步骤3 配置高级选项

- 创建新的网络负载均衡器(Network Load Balancer)
- 设置负载均衡器类型为Network Load Balancer
- 配置负载均衡器名称，此处可自行配置想要的名称，符合AWS命名规范即可，此处设置为**MonoSQL-LB-ALPHA**
- 设置负载均衡器方案为Internal
- 设置可用区与子网为subnet-015bc66fedce1b8c0(公有)
- 设置侦听器与路由监听3306端口，并且默认路由转发到MonoSQL\_LB|TCP

### 4. 步骤4 配置组大小和扩展策略 此处设置vm实例的需求值为3，最大值为8，最小值为0。这里可以按需进行配置

## 配置组大小和扩展策略 - 可选 Info

设置 Auto Scaling 组的所需容量、最小容量和最大容量。您也可以选择添加扩展策略，从而动态扩展组中的实例数量。

### 组大小 - 可选 Info

通过更改所需容量来指定 Auto Scaling 组的大小。您还可以指定最小和最大容量限制。您的所需容量必须在限制范围内。

所需容量

3

最小容量

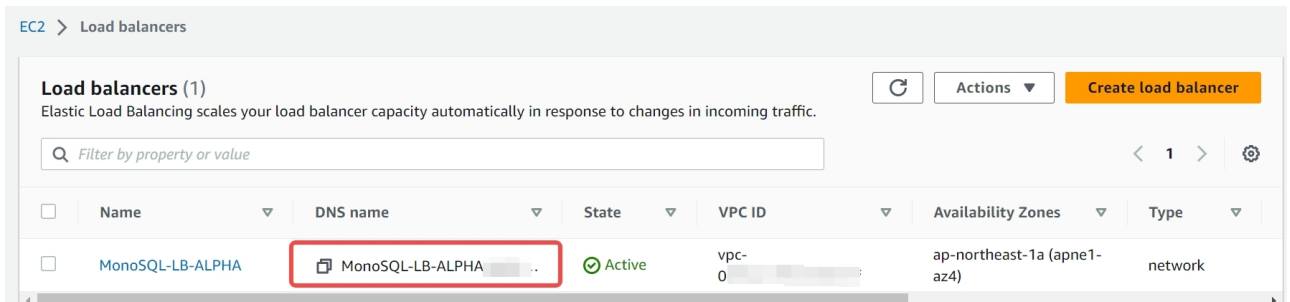
0

最大容量

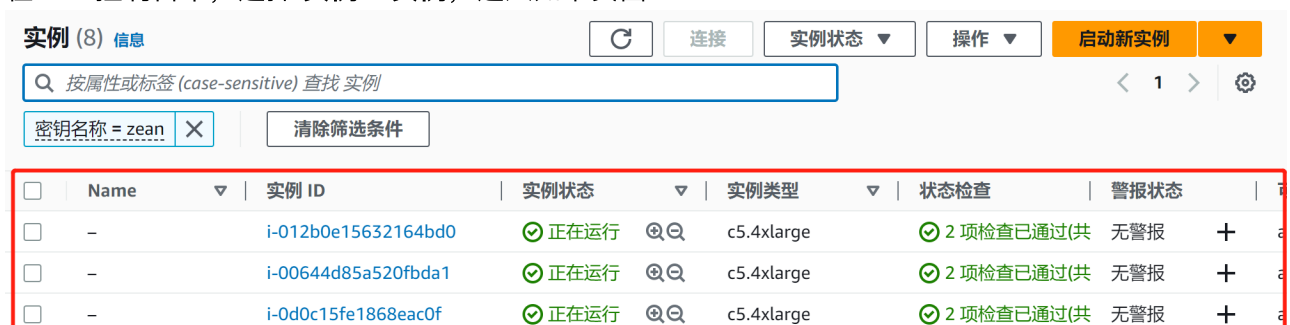
8

上述选项设置完成后，其他步骤设置为默认即可，不需要特别修改。所有的设置完成后，即可成功创建名称为**MonoSQL**的**Auto Scaling**组，随之一同创建成功的还有负载均衡器**MonoSQL-LB-ALPHA**。

- 在AWS控制台中，选择 **负载均衡 > 负载均衡器**，进入如下页面



- 在AWS控制台中，选择 **实例 > 实例**，进入如下页面



可以发现多了三个新的运行实例，这三个实例就是我们在**Auto Scaling**组中设置的3个需求节点。

### 基于MonoSQLMonitor虚拟镜像创建监控实例

MonoSQLMonitor虚拟镜像中集成了Prometheus与Grafana监控组件，可以实现对Auto Scaling Group的监控。Prometheus是一种监控工具，可以收集和存储分布式数据库的指标数据，而Grafana则是一种数据可视化工具，可以将这些指标数据以图表等形式展示出来，帮助用户更好地理解和分析分布式数据库的性能和运行状况。

#### 1. 创建基于MonoSQLMonitor的EC2实例

2. 访问WebUI，查看Grafana监控情况 创建完MonoSQLMonitor监控实例后，该实例默认会监控名称为MonoSQL的Auto Scaling组，您可以在浏览器中输入监控EC2实例ip:3000，查看组件对于该Auto Scaling组的监控。

## 部署选项

### 1. 单Region部署

在特定AWS Region，启动Auto Scaling Group，实例会自动连接当前region的AWS DynamoDB。

### 2. 多Region部署

在多个AWS Region分别启动多个Auto Scaling Group，实例会自动连接到对应region的AWS DynamoDB。设置DynamoDB的表为Global Table，实现数据多写和跨Region自动同步。

## 部署时间

产品在AWS完成部署时间为1个工作日。

## 部署支持的区域

MonoSQL产品支持的AWS区域（Region）包括

1. Asia Pacific (Hong Kong)
2. Asia Pacific (Tokyo)
3. Asia Pacific (Seoul)
4. Asia Pacific (Singapore)
5. US East (N. Virginia)
6. US East (Ohio)
7. US West (N. California)
8. US West (Oregon)

## 产品部署前提要求

### 对部署资源的要求

在AWS上部署产品需要的资源如下：

1. 获取MarketPlace上的MonoSQL AMI，操作系统需使用Ubuntu20.04。
2. 配置AWS AutoScaling group，动态伸缩MonoSQL集群规模
3. AWS DynamoDB权限
4. MonoSQL 监控服务器一台，使用MarketPlace上MonoSQLMonitor AMI。

### 对技能或专业知识要求

熟悉MySQL数据库

熟悉AWS控制台和AWS相关服务

1. Amazon Network Load Balancer
2. Amazon Auto Scaling Group
3. Amazon EC2

4. Amazon VPC
5. Amazon SQS
6. Amazon DynamoDB
7. Amazon Cloud Watch

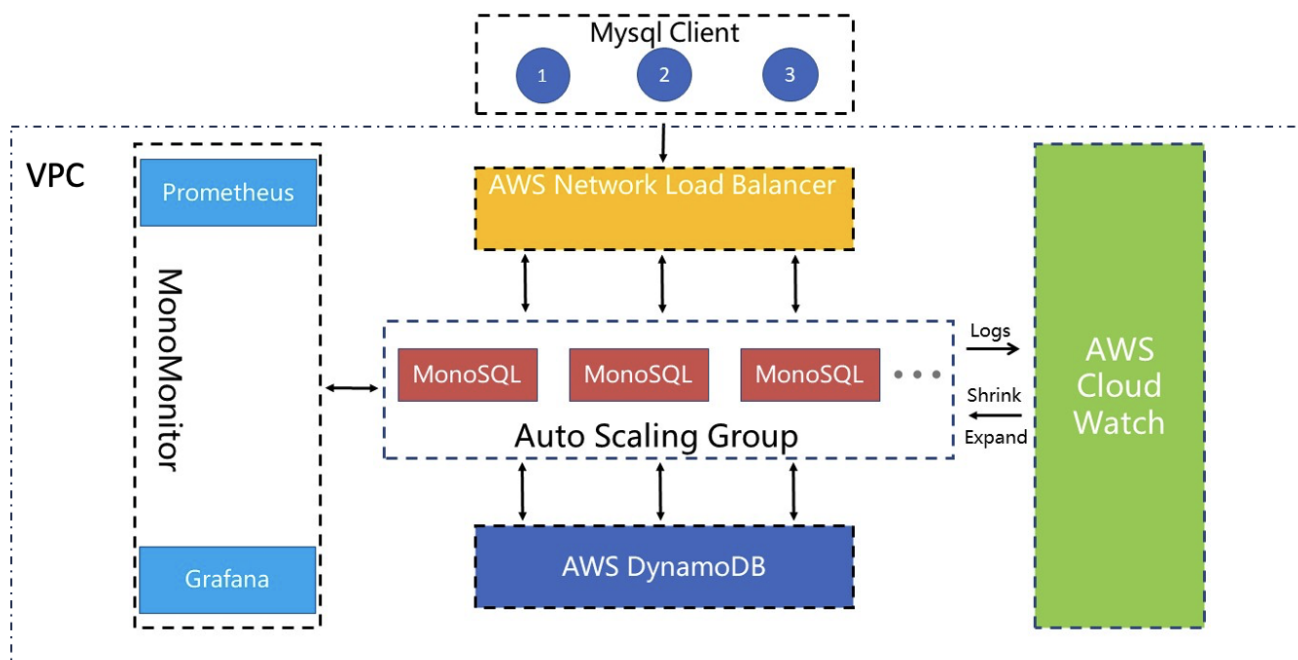
## 系统部署所需的环境配置

客户需要具备AWS账号，如没有，建议联系AWS销售或通过AWS官网进行账号的注册和认证。需使用Ubuntu20.04操作系统进行部署。

## 产品部署架构图

部署产品时通过Amazon Auto Scaling Group实现根据负载动态调整集群规模。

在内核设计上，MonoSQL整体架构可以拆分为多个模块，各个模块之间互相通信，组成一体化的服务、运维、监控系统，对应的架构图如下：



子网信息： MonoSQL计算节点和MonoSQL监控节点应该在同一个VPC和同一个子网内部。

## 产品部署成本预估

### 产品部署项目所需计费服务列表

产品提供下列服务（以下均必选） AWS EC2 with MonoSQL: 推荐c5.4xlarge机型， 80GB gp2存储 AWS Simple Queue Service AWS DynamoDB

### 产品license费用及AWS实价参考

产品无license费用， 计价模型是marketplace 按小时计费， 不同机型的价格不一样， 详见Marketplace页面 以c5.4xlarge为例， 价格为\$0.68/小时软件费用

附EC2官方报价链接 <https://www.amazonaws.cn/en/ec2/pricing/ec2-linux-pricing/>



# 产品部署安全设置

## 安全概述

1. MonoSQL产品不需要使用Root用户进行部署。用户需要拥有EC2FullAccess权限。
2. 本产品没有使用S3等公有资源

## 最低特权策略

用户应遵循AWS最低特权策略来管理用户权限，MonoSQL产品的部署仅需要EC2FullAccess权限。

## 产品需要的IAM角色和权限

DBA需要一个EC2FullAccess权限的用户来管理AutoScaling Group。如果DBA需要访问MonoSQL的其他组件，需要添加对应权限，比如AmazonDynamoDBFullAccess， AmazonSQSFullAccess等。

MonoSQL实例会使用特定的角色`monosql`来访问其他服务，需要AWS服务权限列表如下：

1. CloudWatchLogsFullAccess
2. AmazonDynamoDBFullAccess
3. AmazonEC2FullAccess
4. AmazonSQSFullAccess
5. CloudWatchAgentServerPolicy

## 产品密钥

MonoSQL是无状态计算节点，不需要存储密钥 MonoSQL访问的用户名和密码，存储在DynamoDB中。

## 有关客户敏感数据存储的位置

MonoSQL所有数据存储在DynamoDB中，由DynamoDB提供数据敏感性最佳实践。

详情请参考 <https://aws.amazon.com/blogs/database/applying-best-practices-for-securing-sensitive-data-in-amazon-dynamodb/>

## 数据加密配置

MonoSQL是无状态计算节点，不会存储数据。

数据加密及安全性保证由DynamoDB提供。

详情请参考 <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/EncryptionAtRest.html>

## 网络的相关配置说明及步骤

具体步骤如下：

1. 创建VPC：xxxx
2. 创建子网：10.0.0.0/24段
3. 创建互联网网关：网段 10.168.254.0/24

4. 创建安全组并配置入站及出站规则，开放端口: 3000,3306,3307,9090,9104,9200,18081。

关于IMDSv1

本产品不需要调用AWS API，本产品访问AWS DynamoDB使用AWS Client

产品资源选择

MonoSQL使用EC2实例，用户可以选择不同的实例类型。类型选项包括：

- 1. c5.large
- 2. c5.xlarge
- 3. c5.2xlarge
- 4. c5.4xlarge
- 5. c5.9xlarge
- 6. c5.12xlarge
- 7. c5.24xlarge

产品部署具体步骤

产品型部署架构在 AWS 上搭建的分步说明

引导MonoSQL在DynamoDB中创建系统表

DynamoDB是有AWS提供的NoSQL数据库服务，它可以快速、可靠地存储和检索数据。借助MonoSQL，您可以无痛的从Mysql或者MariaDB的轻松迁移到DynamoDB。在使用MonoSQL进行DynamoDB的访问之前需要先在DynamoDB上创建系统表，系统表是DynamoDB中的一种特殊表格，用于存储与DynamoDB服务本身相关的元数据和配置信息。通过运行引导程序来创建系统表格，可以确保DynamoDB服务能够正常运行并提供所需的功能。

1. 基于MonoSQL的EC2实例创建 选择**EC2 > 实例 > 实例**，进入如下页面



点击**启动新实例**，进入后续的EC2实例具体配置环节。

- 步骤1：配置实例名称，设置为**Mono-Bootstrap**。
- 步骤2：从**AWS Market Place**中查找AMI镜像**MonoSQLServer**，设置AMI为**MonoSQLServer**。
- 步骤3：设置实例类型为**c5.4xlarge**
- 步骤4：设置密钥对，用于ssh免密登陆，可以使用已有密钥对或者创建新密钥对
- 步骤4：配置EC2实例的网络，注意此处设置的安全组必须允许来自3306端口的流量，以进行数据库的访问与设置。完成上述配置后，检查无误后，则可以启动实例。具体设置过程可以查看

如下的演示：



## 2. 在DynamoDB中创建MonoSQL对应的系统表

- 以ssh登陆到新建的EC2实例运行下面的命令，引导MonoSQL在AWS DynamoDB中创建系统表格

```
/home/ubuntu/install/scripts/mysql_install_db --defaults-
file=/home/ubuntu/dynosql.cnf --basedir=/home/ubuntu/install --
datadir=/home/ubuntu/data0 --plugin-
dir=/home/ubuntu/install/lib/plugin > log 2>&1 &
```

```
ubuntu@ubuntu-2023-02-28:~$ /home/ubuntu/install/scripts/mysql_install_db --defau
lts-file=/home/ubuntu/dynosql.cnf --basedir=/home/ubuntu/install --datadir=/hom
e/ubuntu/data0 --plugin-dir=/home/ubuntu/install/lib/plugin > log 2>&1 &
[1] 4039
```

运行`tail -f log`命令查看执行是否成功，如出现`ok`提示，则说明引导成功

```
ubuntu@ubuntu:~$ tail -f log
/home/ubuntu/data0 dir is empty.
Installing MariaDB/MySQL system tables in '/home/ubuntu/data0' ...
2023-04-17 3:03:11 0 [Warning] Plugin 'MONOGRAPH' is of maturity level experim
ental while the server is stable
2023-04-17 3:03:11 0 [Warning] Plugin 'MONOGRAPH_TEMP_TABLE_INFO' is of maturi
ty level experimental while the server is stable
WARNING: Logging before InitGoogleLogging() is written to STDERR
I0417 03:03:19.503041 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-3-0
I0417 03:03:28.915808 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-4-0
I0417 03:03:36.855839 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-6-0
I0417 03:04:02.045787 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-c-0
I0417 03:04:08.275446 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-d-0
I0417 03:04:14.132661 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-f-0
I0417 03:05:13.863559 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-51-0
I0417 03:05:19.145099 4075 ha_monograph.cc:4932] create table succeed/tmp/#sql
-temptable-feb-1-52-0
OK
```

- 查看此时AWS DynamoDB中的表格，可以看到我们通过MonoSQL在AWS DynamoDB中创建的表格，进一步表明MonoSQL引导成功。



- 启动数据库服务

```
/home/ubuntu/install/bin/mysqld --defaults-
file=/home/ubuntu/dynosql.cnf > mysql_log 2>&1 &
```

- 使用sock连接数据库

```
connect to db.
cd ~/install
sudo ./bin/mysql -S /tmp/mysqld3306.sock test
```

- 通过下面的命令创建SQL用户sysb进行基准测试，创建用户mono用于数据库监控。

```
create sysbench user and monitor user.
delete from mysql.user where User='';
CREATE USER 'sysb'@'%' IDENTIFIED BY 'sysb';
GRANT ALL PRIVILEGES ON * . * TO 'sysb'@'%';

CREATE USER IF NOT EXISTS 'mono'@'%' IDENTIFIED BY 'mono' WITH
MAX_USER_CONNECTIONS 5;
GRANT ALL PRIVILEGES ON *.* TO 'mono'@'%' IDENTIFIED BY 'mono'
WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

创建上述角色后，可以进入mysql数据库下，查看user表中，是否存在用户sysb与mono，验证创建是否成功

```
MariaDB [test]> use mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [mysql]> select user from user;
+-----+
| User |
+-----+
| mariadb.sys |
| root |
| mono |
| sysb |
+-----+
4 rows in set (0.095 sec)
```

## 基于MonoSQL创建Auto Scaling Group

Auto Scaling Group是AWS提供的一种云端服务，它允许您自动地调整EC2实例的数量，以满足应用程序的需求。借助Auto Scaling Group，您可以实现根据请求流量自动配置MonoSQL的实例数量，而无需手动干预。选择 **EC2 > Auto Scaling > Auto Scaling组**，进入如下界面 点击**创建Auto Scaling组**，进入后续的Auto

Scaling Group具体配置。

Auto Scaling 组 (5) Info

刷新

编辑

删除

创建 Auto Scaling 组

搜索您的 Auto Scaling 组

| <input type="checkbox"/> | 名称                                   | 启动模板/配置                                  | 实例 | 状态 | 所需容量 | 最... |
|--------------------------|--------------------------------------|------------------------------------------|----|----|------|------|
| <input type="checkbox"/> |                                      | MonoSQLConfZean                          | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | d4c3c020-0197-ff2a-2c3f-02dec84baeb6 | eks-d4c3c020-0197-ff2a-2c3f-02dec84baeb6 | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | 0b66-fd89-7a99-0b5de5088262          | eks-f8c21b8e-0b66-fd89-7a99-0b5de5088262 | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | 07a9-1876-0d59-0bb44ccd3aa7          | eks-c4c20ef1-07a9-1876-0d59-0bb44ccd3aa7 | 0  | -  | 0    | 0    |
| <input type="checkbox"/> | e064-f04c-8e32-9d47dc060b4b          | eks-bcc1acc6-e064-f04c-8e32-9d47dc060b4b | 0  | -  | 0    | 0    |

1. 步骤1 选择启动模板或配置

- 设置Auto Scaling组名称，注意此处的组名称必须设置为MonoSQL，因为后续的监控组件Prometheus与Grafana会依据此名称来对该资源组进行监控。

名称

Auto Scaling 组名称  
输入一个名称来标识该组。  

MonoSQL

名称对于当前区域中的此账户必须是唯一的，且不超过 255 个字符。

- 设置启动模板，启动模板选择启动配置，忽略警告项，选择创建启动配置

启动配置 Info

切换到启动模板

⚠ 我们建议您使用启动模板并使用 Auto Scaling 指导选项，而不是使用启动配置来创建 EC2 自动扩缩组。有关迁移启动配置和使用启动模板的更多信息，[请参阅文档](#)

启动配置  
选择一个包含 Amazon Machine Image (AMI)、实例类型、密钥对和安全组等实例级别设置的启动配置。  

选择启动配置

⚠ 必填

创建启动配置

取消

下一步

按照如下的演示，配置自定义的启动模板

EC2 > 启动配置 > 创建启动配置

### 创建启动配置 信息

 我们建议您使用启动模板并使用自动扩缩指引选项，而不是使用启动配置来创建 EC2 Auto Scaling 组。有关迁移启动配置和使用启动模板的更多信息，[请参阅文档](#) 

[创建启动模板](#)

#### 启动配置名称

名称

MonoSQLConfAlpha

#### Amazon 系统映像 (AMI) 信息

AMI

### 注意

在配置启动模板的时候，设置的IAM实例配置文件需要满足[部署准备](#)中的条件2，即选择已经创建好的名称为**monosql**的用户角色。

#### 其他配置 - 可选

购买选项 信息

☐ 请求 Spot 实例

IAM 实例配置文件 信息

monosql

监控 信息

☐ 启动 CloudWatch 中的 EC2 实例详细监控

2. **步骤2** 选择实例启动选项 选择相应的可用区与子网为ap-northeast-1a | subnet-015bc66fedce1b8c0

## 选择实例启动选项 Info

选择您的实例启动到的 VPC 网络环境，然后自定义实例类型和购买选项。

### 网络 Info

对于大多数应用程序，您可以使用多个可用区，并让 EC2 Auto Scaling 跨可用区平衡实例。默认 VPC 和默认子网适合快速入门。

#### VPC

选择为您的 Auto Scaling 组定义虚拟网络的 VPC。

vpc-0dd5ac199e158931f  
172.31.0.0/16 Default



[创建 VPC](#)

#### 可用区和子网

定义您的 Auto Scaling 组可以在所选 VPC 中使用哪些可用区和子网。

选择可用区和子网



ap-northeast-1a | subnet-015bc66fedce1b8c0 X  
172.31.32.0/20 Default

[创建子网](#)

### 3. 步骤3 配置高级选项

- 创建新的网络负载均衡器(Network Load Balancer)

#### 负载均衡 Info

使用以下选项将 Auto Scaling 组附加到现有负载均衡器或您定义的新负载均衡器。

☐ 无负载均衡器  
Auto Scaling 组的流量前端将不会是负载均衡器。

☐ 附加到现有负载均衡器  
从现有的负载均衡器中进行选择。

☒ 附加到新的负载均衡器  
快速创建基本负载均衡器，以附加到 Auto Scaling 组。

- 设置负载均衡器类型为 Network Load Balancer

#### 负载均衡器类型

请从以下选项中选择负载均衡器类型。创建负载均衡器后，您将无法更改类型选择。如果您需要的负载均衡器类型与此处提供的不同，请访问[负载均衡控制台](#)。

☐ Application Load Balancer  
HTTP、HTTPS

☒ Network Load Balancer  
TCP、UDP、TLS

- 配置负载均衡器名称，此处可自行配置想要的名称，符合AWS命名规范即可，此处设置为 **MonoSQL-LB-ALPHA**
- 设置负载均衡器方案为 Internal

#### 负载均衡器方案

创建负载均衡器后，无法更改架构。

☒ Internal

☐ Internet-facing



- 设置可用区与子网为subnet-015bc66fedce1b8c0(公有)

#### 可用区和子网

您必须为启用的每个可用区选择一个子网。仅公有子网可供选择，以支持 DNS 解析。

☒ ap-northeast-1a

subnet-015bc66fedce1b8c0

☐ ap-northeast-1d

选择子网

☐ ap-northeast-1c

选择子网

- 设置侦听器与路由监听3306端口，并且默认路由转发到MonoSQL\_LB|TCP

#### 侦听器 and 路由

如果您需要安全侦听器或多个侦听器，则可以用以下方式进行配置: [负载均衡控制台](#) 在创建负载均衡器之后，

协议

TCP

端口

3306

默认路由(转发到)

MonoSQL-LB | TCP

4. 步骤4 配置组大小和扩展策略 此处设置vm实例的需求值为3，最大值为8，最小值为0。这里可以按需进行配置

## 配置组大小和扩展策略 - 可选 Info

设置 Auto Scaling 组的所需容量、最小容量和最大容量。您也可以选择添加扩展策略，从而动态扩展组中的实例数量。

### 组大小 - 可选 Info

通过更改所需容量来指定 Auto Scaling 组的大小。您还可以指定最小和最大容量限制。您的所需容量必须在限制范围内。

所需容量

3

最小容量

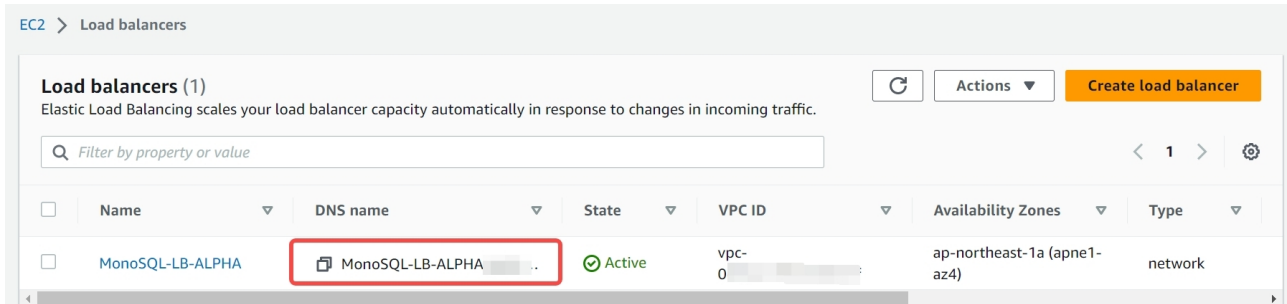
0

最大容量

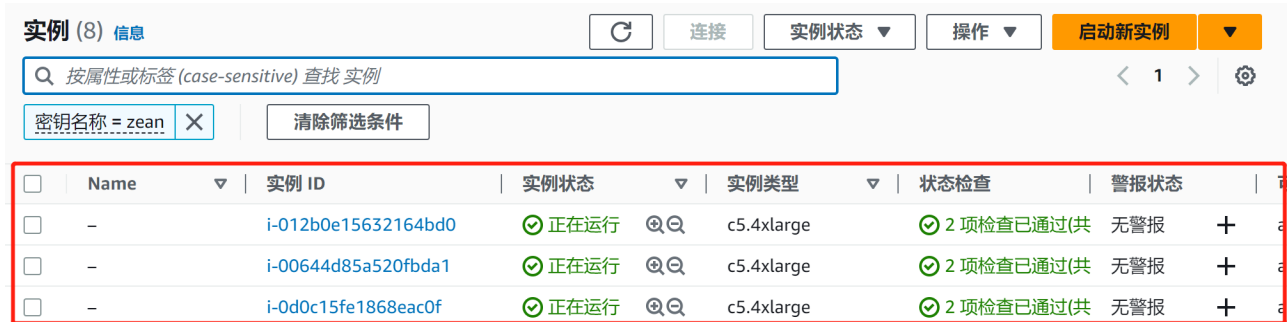
8

上述选项设置完成后，其他步骤设置为默认即可，不需要特别修改。所有的设置完成后，即可成功创建名称为MonoSQL的Auto Scaling组，随之一同创建成功的还有负载均衡器MonoSQL-LB-ALPHA。

- 在AWS控制台中，选择 负载均衡 > 负载均衡器，进入如下页面



- 在AWS控制台中，选择 实例 > 实例，进入如下页面



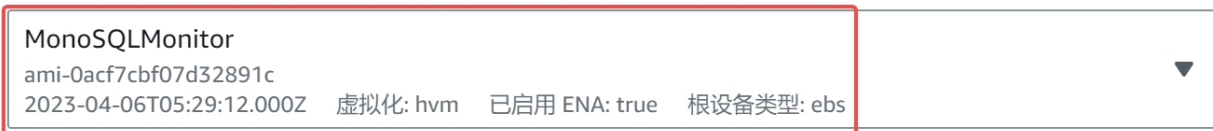
可以发现多了三个新的运行实例，这三个实例就是我们在Auto Scaling组中设置的3个需求节点。

## 基于MonoSQLMonitor虚拟镜像创建监控实例

MonoSQLMonitor虚拟镜像中集成了Prometheus与Grafana监控组件，可以实现对Auto Scaling Group的监控。Prometheus是一种监控工具，可以收集和存储分布式数据库的指标数据，而Grafana则是一种数据可视化工具，可以将这些指标数据以图表等形式展示出来，帮助用户更好地理解和分析分布式数据库的性能和运行状况。

- 创建基于MonoSQLMonitor的EC2实例 创建监控实例的过程与创建MonoSQLServer实例基本一致，需要注意的一点是相应的AMI,需要设置为MonoSQLMonitor。

### Amazon Machine Image (AMI)



### 描述

MonoSQL Monitor with prometheus

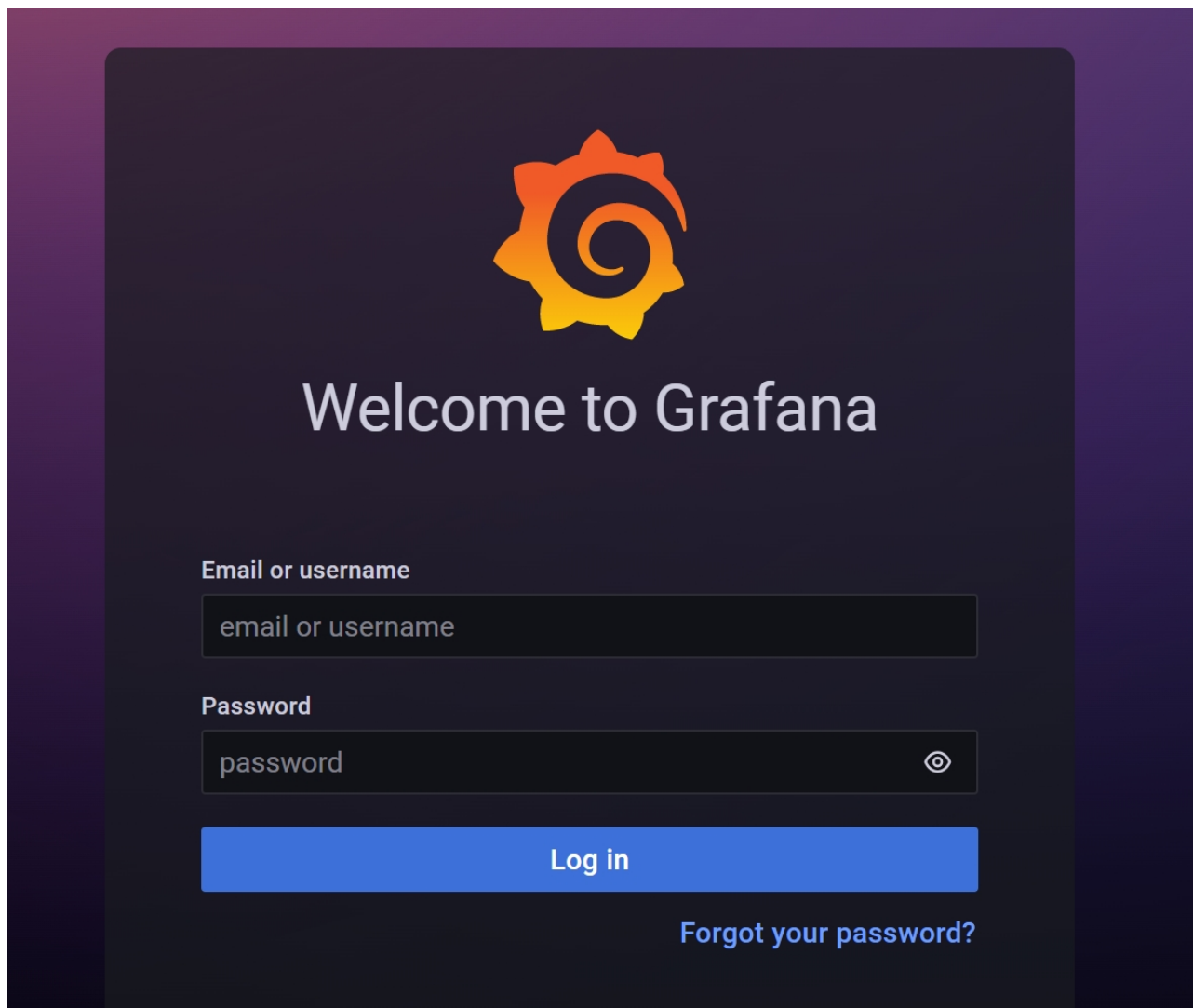
### 架构

### AMI ID

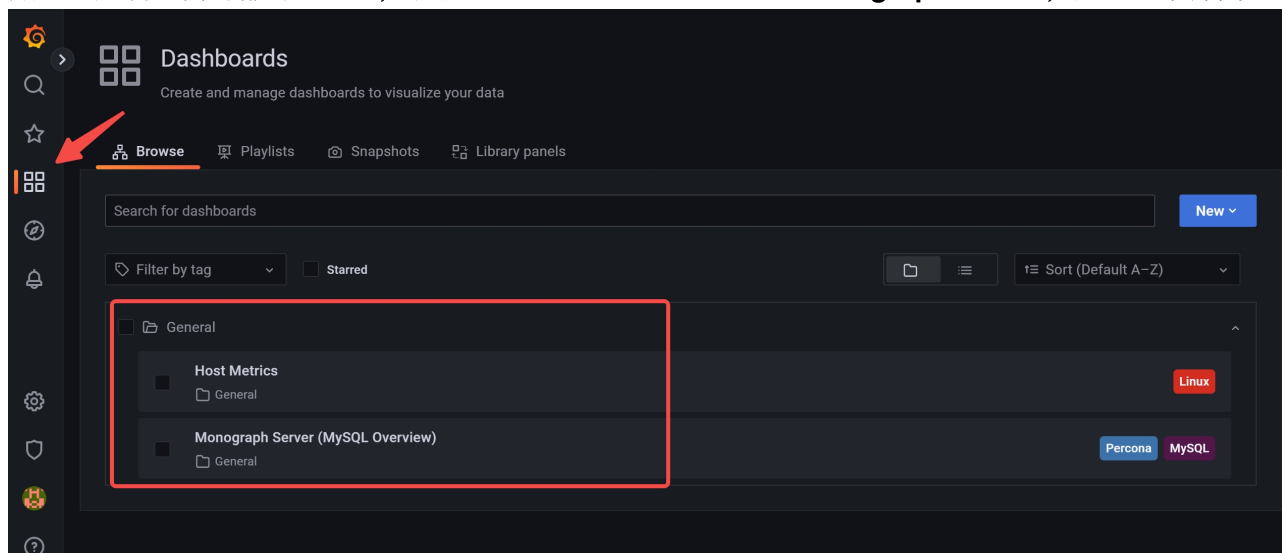
x86\_64

ami-0acf7cbf07d32891c

- 访问WebUI，查看Grafana监控情况 创建完MonoSQLMonitor监控实例后，该实例默认会监控名称为MonoSQL的Auto Scaling组，您可以在浏览器中输入监控EC2实例ip:3000，查看组件对于该Auto Scaling组的监控情况，出现如下界面

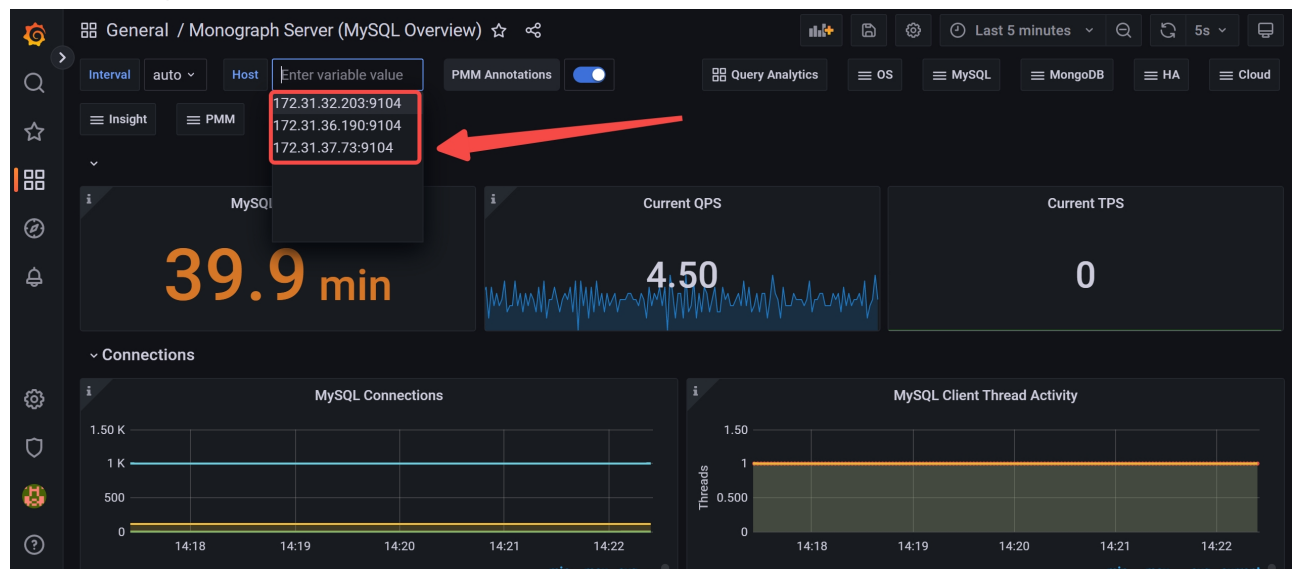


默认用户名与密码都为**admin**， 选择**Dashboards> General > Monograph Server**， 进入如下界面



点击**Monograph Server**， 进入如下界面， 可以看到**Auto Scaling**组下的三个启动实例都处在监控

## 实例的监控中

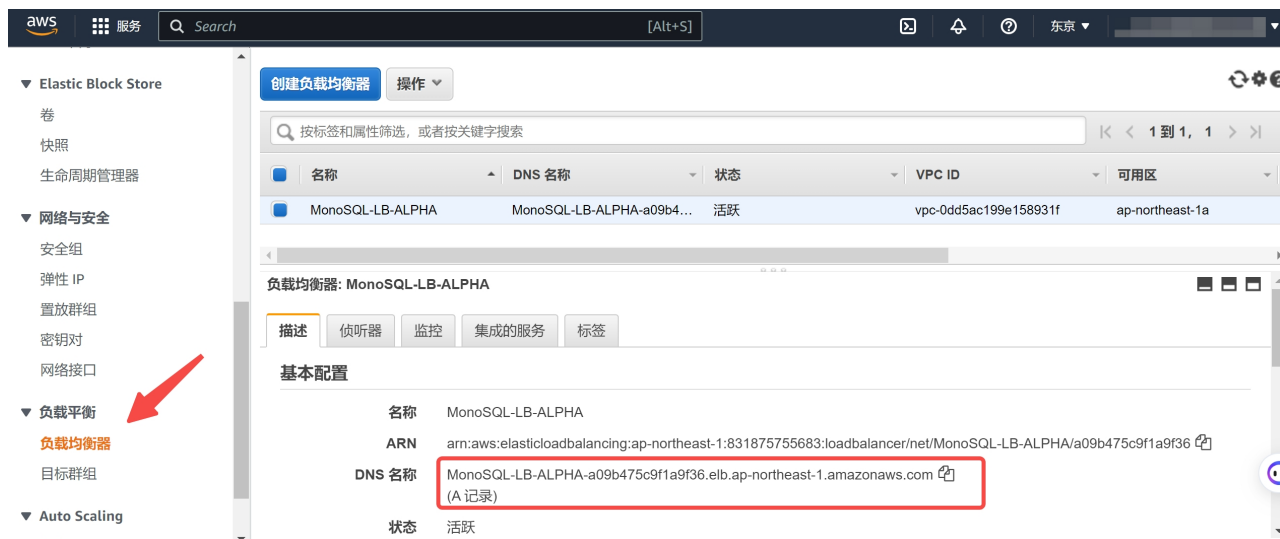


## 使用Sysbench进行基准测试

1. 创建MySQL Client EC2实例 创建MyCleint实例的过程与创建MonoSQLServer实例基本一致，但是需要注意的是相应的AMI，需要设置为**`dynosqlalpha`**。创建该EC2实例的目的在于使得该实例作为MySQL客户端，与我们设置的**Network Load Balancer**进行交互
2. 进行基准测试
  - 登录到创建的MySQL Client实例，执行下述命令进行**Sysbench**测试

```
sysbench /usr/share/sysbench/oltp_insert.lua --
mysql_storage_engine=monograph --tables=1 --table_size=100000 --mysql-
user=sysb --mysql-host=<MonoSQL-dns-example> --mysql-port=3306 --
mysql-password=sysb --mysql-db=test --time=120 --threads=100 --report-
interval=5 --auto_inc=off --create_secondary=false --mysql-ignore-
errors=all prepare
sysbench /usr/share/sysbench/oltp_insert.lua --
mysql_storage_engine=monograph --tables=1 --table_size=100000 --mysql-
user=sysb --mysql-host=<MonoSQL-dns-example> --mysql-port=3306 --
mysql-password=sysb --mysql-db=test --time=120 --threads=300 --report-
interval=5 --auto_inc=off --create_secondary=false --mysql-ignore-
errors=all run
```

将字段**`<MonoSQL-dns-example>`**字段替换为**基于MonoSQL创建Auto Scaling Group**中创建的负载均衡器的实际DNS。



根据我们配置的Auto Scaling Group中的实例数量为3，您可以将线程数`threads`设置为300，下图展示了测试时的运行结果

```
ubuntu@ip-172-31-44-179:~$ sysbench /usr/share/sysbench/oltp_insert.lua --mysql_storage_engine=monograph --tables=1 --
table_size=100000 --mysql-user=sysb --mysql-host=MonoSQL-LB-ALPHA-a09b475c9f1a9f36.elb.ap-northeast-1.amazonaws.com --
mysql-port=3306 --mysql-password=sysb --mysql-db=test --time=120 --threads=300 --report-interval=5 --auto_inc=off --cr
eate_secondary=false --mysql-ignore-errors=all run
sysbench 1.0.18 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 300
Report intermediate results every 5 second(s)
Initializing random number generator from current time

Initializing worker threads...
Threads started!

[5s] thds: 300 tps: 4471.40 qps: 4471.40 (r/w/o: 0.00/4471.40/0.00) lat (ms,95%): 227.40 err/s: 0.00 reconn/s: 0.00
[10s] thds: 300 tps: 3997.02 qps: 3997.02 (r/w/o: 0.00/3997.02/0.00) lat (ms,95%): 204.11 err/s: 0.00 reconn/s: 0.00
[15s] thds: 300 tps: 3961.00 qps: 3961.00 (r/w/o: 0.00/3961.00/0.00) lat (ms,95%): 200.47 err/s: 0.00 reconn/s: 0.00
[20s] thds: 300 tps: 3950.00 qps: 3950.00 (r/w/o: 0.00/3950.00/0.00) lat (ms,95%): 196.89 err/s: 0.00 reconn/s: 0.00
[25s] thds: 300 tps: 3991.60 qps: 3991.60 (r/w/o: 0.00/3991.60/0.00) lat (ms,95%): 200.47 err/s: 0.00 reconn/s: 0.00
```

关于利用Sysbench进行基准测试的详细解释与过程，参考[MonoSQL Benchmark Result](#)

## 产品备份和恢复

MonoSQL产品无状态，备份和恢复请参考DynamoDB文档 <https://aws.amazon.com/dynamodb/backup-restore/>

## 产品日常维护

### 产品系统凭证和加密密钥的分步说明

MonoSQL是AWS Marketplace的AMI，需要通过AMI role进行绑定，用于访问AWS DynamoDB，AWS SQS，AWS CloudWatch等服务。

AMI role的管理可通过AWS IAM界面。

启动AWS AutoScaling Group需要特定AWS用户，该用户的密钥管理步骤如下：

1. 访问我的安全凭证,选择用户，点击要更新的用户。
2. 选择安全证书选项，点击停用 访问密钥，然后选择删除密钥。
3. 创建访问密钥，下载密钥。更新密钥完成。

## 软件补丁和升级

MonoSQL产品是无状态服务，用户只需要将当前AWS AutoScaling Group的期望实例数设置为0，之后重新设置启动模版AMI，选取MonoSQL的新版本即可。

## 平台管理许可证的简要说明

MonoSQL不支持BYOD(Bring your own license)模式，用户无需购买和管理软件许可。

用户可以通过按小时计费模式按需启动MonoSQL计算实例。

## AWS服务限制规范性指导

MonoSQL会使用AWS EC2，DynamoDB等服务，这些服务都会涉及AWS服务配额。

AWS 为每个账户维护服务配额（以前称为服务限制），以帮助确保 AWS 资源的可用性并防止意外预置超出需要的资源。有些服务配额会在您使用 AWS 时随着时间自动提高。但是，大多数 AWS 服务需要您手动请求提高配额。

具体请参阅链接 <https://repost.aws/zh-Hans/knowledge-center/manage-service-limits>

## 产品紧急维护

### 产品故障处理分步说明

公司提供Support支持，可以通过电子邮件，在线会议和线下等方式解决故障问题。

1. 针对MonoSQL计算节点问题，由公司Support团队负责。
2. 针对数据存储问题，可通过AWS Support渠道，进行DynamoDB相关问题反馈。
3. 针对监控系统问题，由公司Support团队负责。

### 恢复软件分步说明

1. 针对MonoSQL计算节点问题，因为计算节点无状态，用户只需要关闭错误节点，通过AutoScaling Group自动拉起新计算节点。
2. 针对数据存储，DynamoDB支持跨Region高可用，数据恢复对用户完全透明。

## 产品售后支持

产品售后维护保障解决方案，主要包括邮件技术支持服务、远程技术支持服务、现场技术支持服务等，帮助客户维护更加稳定、高效的业务运行环境。

1. 邮件技术支持服务：交付后，系统进入维护期，操作人员在操作过程中出错并影响业务运行的问题可以直接发送邮件到AWSMarketplace@monographdb.com。我司安排有经验的工程师值班。当设备出现故障时，可通过该途径进行邮件故障申报。
2. 远程技术支持服务：

对于通过邮件指导不能解决的故障，我司在征得甲方同意后，通过远程接入手段，登陆到故障设备，进行故障诊断，查找故障原因，指导现场维护人员处理故障。

要求：我司工程师登陆到故障设备，通过诊断，分析故障产生的原因，指定故障解决技术方案后，并将技术方案通过电话、邮件、微信群等方式通知甲方，经审批后，才能进行故障解决方案的具体措施。在远程登陆过程中，我司工程师通过远程发送的任何指令，同命令日志文件一同整理进故障报告中。

### 3. 现场技术支持服务：

对于通过邮件技术支持和远程技术支持都不能解决的故障问题，我司迅速提供现场支持服务，安排经验丰富的技术支持工程师赴现场分析故障原因，制定故障解决方案，并最终排除故障。

### 4. 联系方式：

邮箱：AWSMarketplace@monographdb.com